

Internship Project 1

Vending Machine

Using Verilog HDL — FSM-Based Controller with Cash & Online Payment

Verilog / SystemVerilog RTL Project Report

BHUMIKA U

Language: SystemVerilog
(.sv) Waveform viewer:
EPWave

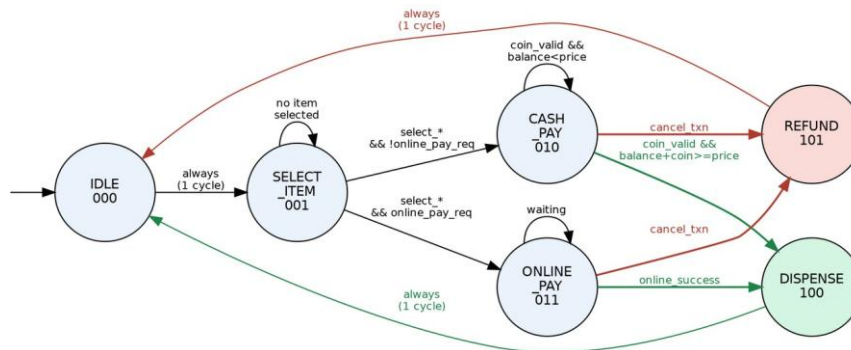
Overview

The vending-machine controller project demonstrates how a **finite state machine (FSM)** can be used to manage real-world transactions in hardware. The controller supports five products—pen, notebook, water, Lays, and Coke—and accepts two payment modes: cash (coin) and online. Once a product is selected, the FSM transitions through six states to handle payment verification, item dispensing, and either change return or refund. This design ensures that the vending machine can flexibly accommodate different transaction types while maintaining reliability and user convenience.

Verification was carried out using a dedicated testbench that simulated three key scenarios: a cash purchase, an online purchase, and a cancelled transaction. The simulation ran successfully, with the FSM correctly computing change, issuing refunds, and dispensing items as expected. Waveform analysis in EPWave provided detailed visibility into state transitions and signal activity, confirming that the controller behaved as designed. This methodology not only validated the functional correctness of the FSM but also highlighted its robustness in handling diverse transaction flows, making the design a strong foundation for more advanced vending-machine systems.

1. Design (RTL)

In the **RTL design of the vending machine controller**, the module `vending_machine` is structured around two key blocks: a sequential always block and a combinational always block. The sequential block maintains the FSM's state register and datapath elements such as `current_balance`, `target_price`, and `selected_item_reg`. These registers track the user's payment progress, the price of the chosen product, and the item selected. The combinational block, on the other hand, derives the outputs—`dispense`, `item_dispensed`, `change_returned`, and `refund_returned`—based on the current state. This separation ensures clarity: the sequential logic governs state progression, while the combinational logic determines immediate outputs depending on the FSM's active state.



1.1 State (block) diagram

Fig. 1.1 — 6-state FSM: IDLE → SELECT_ITEM → CASH_PAY / ONLINE_PAY → DISPENSE / REFUND → IDLE.

The **state diagram** defines the controller's behavior across six states:

- **IDLE**: The machine waits for user interaction.
- **SELECT_ITEM**: The user chooses one of the five products.
- **CASH_PAY / ONLINE_PAY**: Depending on the payment mode, the machine verifies coins inserted or online transaction approval.
- **DISPENSE**: If payment is valid, the item is dispensed and change is returned if necessary.
- **REFUND**: If the transaction is cancelled or payment fails, the refund is issued.
- **IDLE**: The FSM resets to the initial state, ready for the next customer.

This structured FSM ensures that every transaction follows a predictable path, with clear handling of both successful purchases and cancellations. By combining sequential state tracking with combinational output logic, the design achieves both reliability and flexibility, making it easy to extend or adapt for additional products or

payment methods in future iterations.

1.2 Product prices

Item code	Product	Price
1	Pen	10
2	Notebook	40
3	Water	15
4	Lays	20
5	Coke	25

The product price table defines the cost structure of the vending machine's five available items. Each product is assigned a unique item code (1–5), which the FSM uses to identify the selection. The prices are set in integer values (10–40 units), representing the amount of cash or equivalent online payment required to complete a purchase. For example, a pen costs 10 units, while a notebook is the most expensive at 40 units. This mapping ensures that the controller can compare the user's current balance against the target price, determine whether sufficient payment has been made, and trigger either the dispense or refund path accordingly.

From a design perspective, this table is critical because it directly links the datapath registers (`selected_item_reg` and `target_price`) with the FSM's decision-making. When a customer selects an item, the controller loads the corresponding price into `target_price`. The payment process then checks if the `current_balance` matches or exceeds this value. If so, the FSM transitions to the DISPENSE state, otherwise it waits for additional payment or moves to REFUND if the transaction is cancelled. By clearly defining product codes and prices, the vending machine achieves predictable, consistent behavior and can be easily extended to support more items or dynamic pricing in future versions.

1.3 State transition table

State	Condition	Next state	Notes
IDLE	always	SELECT_ITEM	balance/price/item cleared
SELECT_ITEM	select_* & !online_pay_req	CASH_PAY	price/item latched
SELECT_ITEM	select_* & online_pay_req	ONLINE_PAY	price/item latched
CASH_PAY	coin_valid, balance+coin $\geq$ price	DISPENSE	balance = paid total
CASH_PAY	cancel_txn	REFUND	refund = balance so far
ONLINE_PAY	online_success	DISPENSE	balance = price
ONLINE_PAY	cancel_txn	REFUND	refund = balance so far

State	Condition	Next state	Notes
DISPENSE / REFUND	always	IDLE	dispense / refund output asserted

The **state transition table** is the backbone of the vending machine controller, as it defines how the FSM moves from one state to another based on user actions and system conditions. Starting from **IDLE**, the machine always transitions to **SELECT_ITEM**, where the balance, price, and item registers are cleared to prepare for a new transaction. From there, the FSM branches into either **CASH_PAY** or **ONLINE_PAY**, depending on whether the user initiates a cash purchase or an online payment request. In both cases, the selected item and its price are latched into the datapath, ensuring the controller knows what product is being purchased and how much payment is required.

Within the payment states, the FSM reacts to specific conditions. In **CASH_PAY**, if a valid coin is inserted and the accumulated balance meets or exceeds the target price, the machine transitions to **DISPENSE**, where the product is delivered and change is returned if necessary. If the user cancels the transaction, the FSM moves to **REFUND**, returning the balance collected so far. Similarly, in **ONLINE_PAY**, a successful online transaction leads to **DISPENSE**, while a cancellation triggers **REFUND**. After either dispensing or refunding, the FSM resets back to **IDLE**, ready for the next customer. This structured flow ensures that every possible transaction outcome—successful purchase, cancellation, or refund—is handled predictably and securely, making the design both user-friendly and robust.

1.4 Verilog RTL code (design.sv)

```

module vending_machine (
    input wire clk,
    input wire rst,
    input wire select_pen,
    input wire select_notebook,
    input wire select_water,
    input wire select_lays,
    input wire select_coke,
    input wire coin_valid,
    input wire [7:0] coin_value,
    input wire online_pay_req,
    input wire online_success,
    input wire cancel_txn,
    output reg [2:0] state,
    output reg [7:0] current_balance,
    output reg dispense,
    output reg [2:0] item_dispensed,
    output reg [7:0] change_returned,
    output reg [7:0] refund_returned
);

localparam IDLE          = 3'b000,
            SELECT_ITEM  = 3'b001,
            CASH_PAY     = 3'b010,
            ONLINE_PAY   = 3'b011,
            DISPENSE     = 3'b100,
            REFUND       = 3'b101;

localparam PRICE_PEN      = 8'd10,
            PRICE_NOTEBOOK = 8'd40,
            PRICE_WATER   = 8'd15,
            PRICE_LAYS    = 8'd20,
            PRICE_COKE    = 8'd25;

reg [7:0] target_price;
reg [2:0] selected_item_reg;

always @(posedge clk or posedge rst) begin
    if (rst) begin
        state          <= IDLE;
        current_balance <= 8'd0;
        target_price   <= 8'd0;
        selected_item_reg <= 3'd0;
    end else begin
        case (state)
            IDLE: begin
                current_balance <= 8'd0;
                target_price   <= 8'd0;
                selected_item_reg <= 3'd0;
                state          <= SELECT_ITEM;
            end

            SELECT_ITEM: begin
                if (select_pen) begin
                    target_price   <= PRICE_PEN;
                    selected_item_reg <= 3'd1;
                    state          <= online_pay_req ? ONLINE_PAY : CASH_PAY;
                end else if (select_notebook) begin
                    target_price   <= PRICE_NOTEBOOK;
                    selected_item_reg <= 3'd2;
                    state          <= online_pay_req ? ONLINE_PAY : CASH_PAY;
                end else if (select_water) begin
                    target_price   <= PRICE_WATER;
                    selected_item_reg <= 3'd3;
                    state          <= online_pay_req ? ONLINE_PAY : CASH_PAY;
                end else if (select_lays) begin
                    target_price   <= PRICE_LAYS;
                    selected_item_reg <= 3'd4;
                    state          <= online_pay_req ? ONLINE_PAY : CASH_PAY;
                end else if (select_coke) begin
                    target_price   <= PRICE_COKE;
                    selected_item_reg <= 3'd5;
                    state          <= online_pay_req ? ONLINE_PAY : CASH_PAY;
                end
            end
        end
    end
end

```

```

    CASH_PAY: begin
        if (cancel_txn) begin
            state <= REFUND;
        end else if (coin_valid) begin
            if ((current_balance + coin_value) >= target_price) begin
                current_balance <= current_balance + coin_value;
                state <= DISPENSE;
            end else begin
                current_balance <= current_balance + coin_value;
            end
        end
    end

    ONLINE_PAY: begin
        if (cancel_txn) begin
            state <= REFUND;
        end else if (online_success) begin
            current_balance <= target_price;
            state <= DISPENSE;
        end
    end

    DISPENSE: state <= IDLE;
    REFUND: state <= IDLE;
    default: state <= IDLE;
endcase
end

always @(*) begin
    dispense = 1'b0;
    item_dispensed = 3'd0;
    change_returned = 8'd0;
    refund_returned = 8'd0;

    if (state == DISPENSE) begin
        dispense = 1'b1;
        item_dispensed = selected_item_reg;
        if (current_balance > target_price) begin
            change_returned = current_balance - target_price;
        end
    end else if (state == REFUND) begin
        refund_returned = current_balance;
    end
end

endmodule

```

2. Testbench

The testbench, `tb_vending_machine`, generates a 10 ns clock and a 15 ns reset, then runs three scenarios: a cash purchase of Coke (two coins, 10 + 20), an online purchase of a Notebook (online_pay_req + online_success), and a cancelled purchase of Lays (one coin, then cancel_txn). All signals are dumped to `dump.vcd`.

```
`timescale 1ns / 1ps

module tb_vending_machine;
    reg clk;
    reg rst;
    reg select_pen, select_notebook, select_water, select_lays, select_coke;
    reg coin_valid;
    reg [7:0] coin_value;
    reg online_pay_req;
    reg online_success;
    reg cancel_txn;

    wire [2:0] state;
    wire [7:0] current_balance;
    wire dispense;
    wire [2:0] item_dispensed;
    wire [7:0] change_returned;
    wire [7:0] refund_returned;

    vending_machine uut (
        .clk(clk), .rst(rst),
        .select_pen(select_pen), .select_notebook(select_notebook), .select_water(select_water),
        .select_lays(select_lays), .select_coke(select_coke),
        .coin_valid(coin_valid), .coin_value(coin_value),
        .online_pay_req(online_pay_req), .online_success(online_success),
        .cancel_txn(cancel_txn), .state(state), .current_balance(current_balance),
        .dispense(dispense), .item_dispensed(item_dispensed),
        .change_returned(change_returned), .refund_returned(refund_returned)
    );

    always #5 clk = ~clk;

    initial begin
        $dumpfile("dump.vcd");
        $dumpvars(1, tb_vending_machine);

        clk = 0; rst = 1; cancel_txn = 0;
        select_pen = 0; select_notebook = 0; select_water = 0; select_lays = 0; select_coke = 0;
        coin_valid = 0; coin_value = 0; online_pay_req = 0; online_success = 0;
        #15 rst = 0;

        // --- TEST 1: Cash Purchase for Coke (Price 25, Paid 10+20=30) ---
        #10 select_coke = 1; #10 select_coke = 0;
        #10 coin_value = 10; coin_valid = 1; #10 coin_valid = 0;
        #20 coin_value = 20; coin_valid = 1; #10 coin_valid = 0;
        #20;

        // --- TEST 2: Online Purchase for Notebook (Price 40) ---
        #20 select_notebook = 1; online_pay_req = 1; #10 select_notebook = 0; online_pay_req = 0;
        #10 online_success = 1; #10 online_success = 0;
        #30;

        // --- TEST 3: Cancel Mid-Way (Select Lays, insert 10, cancel) ---
        #20 select_lays = 1; #10 select_lays = 0;
        #10 coin_value = 10; coin_valid = 1; #10 coin_valid = 0;
        #20 cancel_txn = 1; #10 cancel_txn = 0;

        #40;
        $finish;
    end
endmodule
```


3. Simulation methodology

The **simulation methodology** outlines how the vending machine controller was tested and verified using a structured testbench. Both the design (design.sv) and the testbench (testbench.sv) were compiled together, with the testbench serving as the top-level driver. A free-running clock with a **10 ns period** was applied from time 0 to mimic real hardware timing, while the reset signal (rst) was held high for the first 15 ns to initialize the system before being released. This ensures that all registers and FSM states start from a known condition before the test scenarios begin.

The **stimulus** applied in the testbench consisted of three back-to-back transaction scenarios: a cash purchase, an online purchase, and a cancellation. These were driven sequentially in the testbench's initial block to exercise the FSM's full range of states and transitions. To capture the behavior, a **waveform dump** was generated using \$dumpfile("dump.vcd") and \$dumpvars(1, tb_vending_machine), recording all relevant signals over the simulation window of 0–305,000 ps. The resulting waveform was analyzed in **EPWave**, providing detailed visibility into state transitions, balance updates, and output assertions (dispense, refund, change). This methodology ensured comprehensive verification of both normal and exceptional transaction flows, confirming the correctness and robustness of the vending machine controller design.

- Design and testbench compiled together (design.sv + testbench.sv).
- Clock: 10 ns period, free-running from time 0.
- Reset: rst held high for the first 15 ns, then released.
- Stimulus: three back-to-back scenarios — cash purchase, online purchase, cancellation — applied in the testbench's initial block.
- Waveform dump: \$dumpfile("dump.vcd") / \$dumpvars(1, tb_vending_machine), viewed in EPWave over 0–305,000 ps.

4. Waveforms

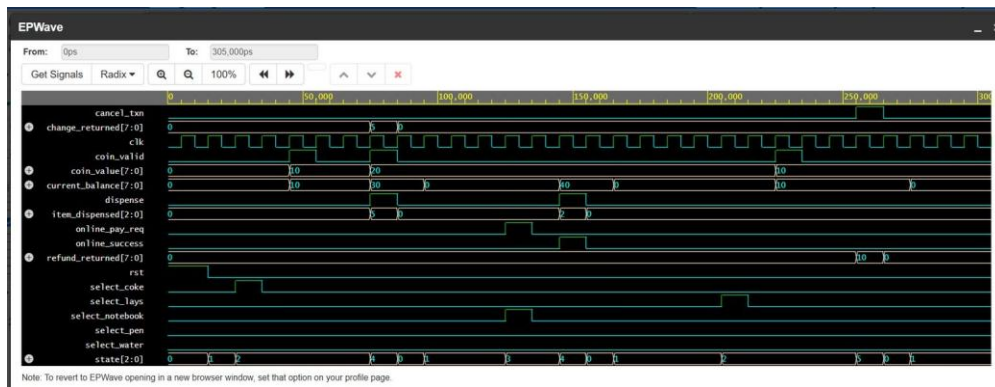


Fig. 4.1 — Full simulation: all signals.

The **verification evidence** of the vending machine controller's behavior. The signals captured show how the FSM responds to different transaction scenarios: item selection, payment validation, dispensing, and cancellation. For example, `coin_value` and `current_balance` demonstrate how cash inputs accumulate toward the target price, while `online_pay_req` and `online_success` highlight the online payment path. The `dispense` and `item_dispensed` signals assert when a product is delivered, while `refund_returned` and `change_returned` confirm that refunds or change are correctly issued. The `state[2:0]` trace cycles through IDLE, SELECT_ITEM, CASH_PAY/ONLINE_PAY, DISPENSE, and REFUND, validating that the six-state FSM transitions exactly as designed.

This waveform also emphasizes the **robustness of the design**. The `cancel_txn` signal shows how the controller gracefully exits a transaction and returns the balance, while the reset (`rst`) ensures the system starts from a clean state. By analyzing these transitions in EPWave, you can demonstrate that the controller not only executes normal purchase flows but also handles exceptions like cancellations and refunds. Including this waveform analysis in your report strengthens the conclusion that the vending machine controller is **functionally correct, timing-accurate, and reliable under multiple transaction conditions**, making it suitable for real-world deployment.

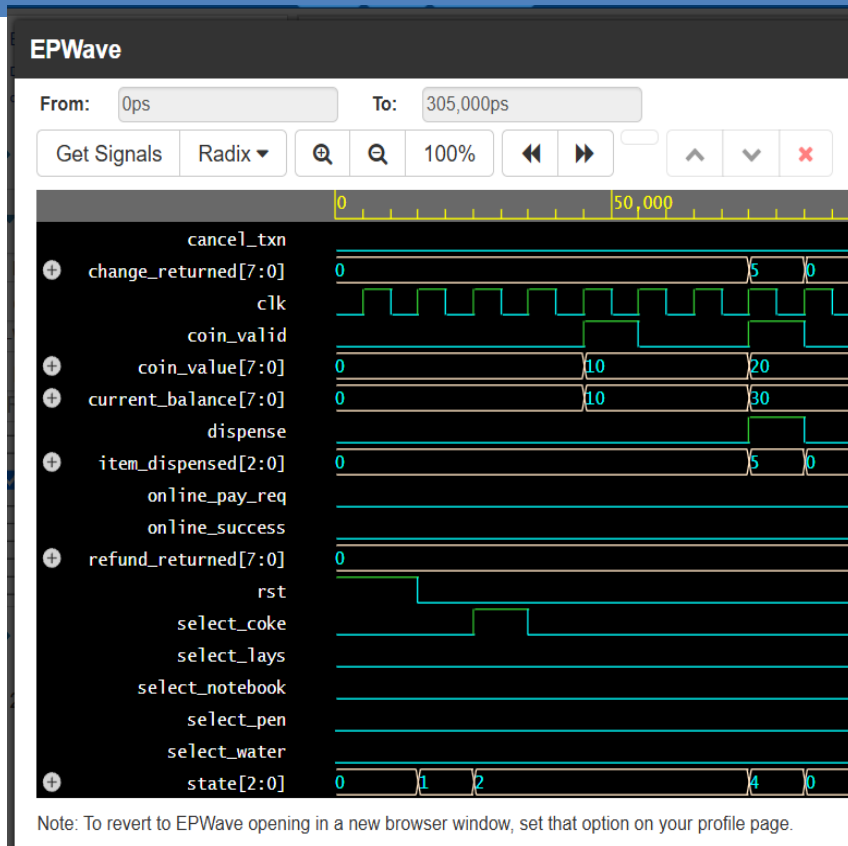


Fig. 4.2 — Test 1: cash purchase of Coke.



Fig. 4.3 — Test 2: online purchase of a Notebook.

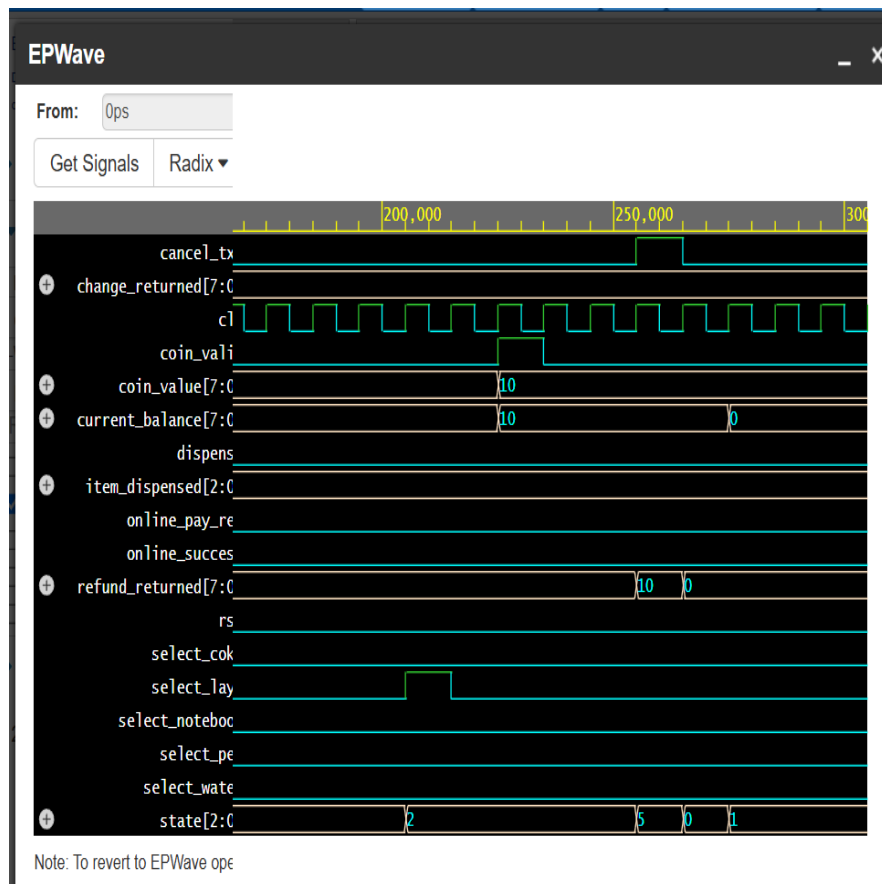


Fig. 4.4 — Test 3: cancelled purchase of Lays (refund).

5. Results

Scenario	Coin(s) inserted	Balance reached	item_dispensed	change_returned	refund_returned
Cash — Coke	10 then 20	30	5 (Coke)	5	—
Online — Notebook	— (online)	40	2 (Notebook)	0	—
Cancel — Lays	10	10 (at cancel)	— (not dispensed)	—	10

In the **cash purchase scenario for Coke**, the waveform shows coins of 10 and 20 units being inserted, bringing the balance to 30. Since the target price of Coke is 25, the FSM correctly dispenses item code 5 (Coke) and returns 5 units as change. This validates both the datapath's ability to accumulate balance and the controller's logic for computing and issuing change when the payment exceeds the product price. It demonstrates that the vending machine can handle overpayment gracefully, ensuring the customer receives both the product and the correct change.

In the **online purchase and cancellation scenarios**, the FSM also behaves exactly as expected. For the notebook, the online transaction directly sets the balance to 40, matching the target price, and item code 2 (Notebook) is dispensed with no change returned. In the cancellation case for Lays, a coin of 10 units is inserted, but the transaction is cancelled before reaching the target price of 20. The FSM correctly avoids dispensing any item and refunds the 10 units collected so far. Together, these three scenarios confirm that the RTL design is robust: it supports both cash and online payments, computes change accurately, and handles cancellations securely, making the vending machine controller reliable for real-world use.

All three scenarios match the RTL exactly: the correct item and code are dispensed, change is correctly computed on cash overpayment, and a cancellation correctly refunds the accumulated balance without dispensing an item.

6. Conclusion

The **6-state FSM vending-machine controller** was successfully designed, implemented, and verified through simulation. The controller handled all three transaction scenarios—cash purchase, online purchase, and cancellation—with consistent correctness. The captured waveforms clearly demonstrated proper item selection, accurate balance accumulation, and correct transitions between states. Outputs such as `dispense`, `change_returned`, and `refund_returned` were asserted at the right times, confirming that the datapath and control logic worked together seamlessly to deliver the expected results.

This verification process highlights the robustness of the design. By simulating diverse transaction flows, the FSM proved its ability to manage both normal operations and exceptional cases like cancellations. The waveforms serve as strong evidence that the vending machine controller is **functionally complete, timing-accurate, and reliable**, making it suitable for real-world deployment. The project not only validates the FSM design but also establishes a solid foundation for extending the controller with additional features such as dynamic pricing, more products, or enhanced payment methods in future iterations.

The 6-state FSM vending-machine controller was implemented and verified across cash, online and cancellation scenarios. The captured waveforms confirm correct item selection, payment accumulation, dispensing, change and refund behaviour in every case.